Cómputo Concurrente 2026
Práctica 4: Spinlocks y algunas primitivas

Montes de Oca Aguilar Yessica Vianney

1. Descripción de los programas entregados

Los programas proporcionados están en la carpeta principal de la práctica y el programa implementado está en la carpeta `ejercicios`

Todos los archivos pertenecen al paquete `unam.fc.concurrent.practica4` y compilan con Java 21 LTS desde la carpeta `practica4`:

```
javac --release 21 -d out unam/fc/concurrent/practica4/*.java unam/fc/concurrent/practica4/
    ejercicios/*.java
java -cp out unam.fc.concurrent.practica4.RunSpin
java -cp out unam.fc.concurrent.practica4.ejercicios.RunSpinMatrix
```

Programas proporcionados

`TASLock`, `TTASLock`, `BackoffLock`, `MCSLock`, `ALock`, `CLHLock`
: Spinlocks del libro de Herlihy y Shavit. Implementan la interfaz `Lock` usando primitivas atómicas (`getAndSet`, `getAndIncrement`, `compareAndSet`). `Backoff` es la clase auxiliar que usa `BackoffLock` para dormir un tiempo aleatorio, cada vez mayor, cuando no obtiene el candado.

`CounterAtomic` **(Programa 1)**
: Contador linealizable no bloqueante. No usa candados: `increment()` es un solo `getAndIncrement()` atómico y `getValue()` un `get()`. Se usó sin modificar como una columna más de la tabla del Ejercicio 1, para comparar los candados contra una solución que no necesita exclusión mutua.

`RunSpin` **(Programa 2) Ejercicio 1**
: Envía k tareas a un `ExecutorService` con h hilos; cada tarea incrementa un contador compartido dentro de la sección crítica protegida por el candado, y al final se imprime el tiempo total en milisegundos. Para llenar la tabla se cambiaron a mano, en cada ejecución, el candado (se descomenta una sola línea `Lock lock = ...`), el número de tareas (límite del `for`) y el número de hilos (`numberThreads`). Para medir `CounterAtomic` se usa la línea `counter.increment()` en lugar de `task(lock)`.

Programa implementado (está en la carpeta `ejercicios`)

`RunSpinMatrix` – **Ejercicio 2**
: Versión de `RunSpin` con la función `task(lock)` modificada: en lugar de incrementar un contador, cada hilo, dentro de la sección crítica, asigna valores a una matriz compartida de 10×10 (función `asignarMatriz`) y después imprime su contenido (función `imprimirMatriz`). La tarea número id escribe $id \cdot 1000 + 10i + j$ en la casilla (i, j), de modo que en la salida se puede comprobar que cada matriz impresa pertenece a una sola tarea, es decir, que el candado garantiza la exclusión mutua. El candado y el número de hilos se cambian a mano igual que en `RunSpin`.

Modificación de `task(lock)`

```
static final int N = 10;
static int[][] matrix = new int[N][N]; // matriz compartida

// Asigna valores a la matriz
public static void asignarMatriz(int id) {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            matrix[i][j] = id * 1000 + i * N + j;
}

// Imprime el contenido de la matriz
public static void imprimirMatriz(int id) {
    StringBuilder sb = new StringBuilder();
    sb.append("Tarea ").append(id).append('\n');
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) sb.append(String.format("%7d", matrix[i][j]));
        sb.append('\n');
    }
    System.out.print(sb);
}

// task(lock) modificada
private static int task(Lock lock, int id) {
    lock.lock();
    try {
        asignarMatriz(id); // asignación dentro de la sección crítica
        imprimirMatriz(id); // impresión una vez completada la asignación
    } finally {
        lock.unlock();
    }
    return id;
}
```

La impresión también se hace dentro de la sección crítica, si se hiciera fuera otro hilo podría sobrescribir la matriz mientras se imprime y se verían matrices mezcladas. En todas las ejecuciones cada matriz impresa contuvo únicamente valores de su propia tarea (por ejemplo la tarea 88 imprime $88000, \ldots, 88099$) aunque las tareas aparecen en desorden porque el orden de entrada depende de qué hilo obtiene primero el candado.

2. Mediciones

La práctica se realizó de manera individual y la omputadora utilizada: **12 hilos**, por lo que `max-1` $= 11$. Cada casilla corresponde a una ejecución del programa, tiempos en milisegundos.

Ejercicio 1

Configuración	TAS	TTAS	Backoff	MCS	ALock	CLH	Reentrant	Counter Atomic
400 tareas, 4 hilos	9.54	6.58	28.89	8.49	7.54	7.77	6.57	7.40
1000 tareas, 4 hilos	12.19	8.15	26.16	9.48	9.45	12.25	12.80	7.92
1000 tareas, 11 hilos	9.02	13.90	34.20	10.55	18.64	9.48	9.40	9.23
Promedio	10.25	9.54	29.75	9.51	11.88	9.83	9.59	**8.18**

Cuadro 1: Tiempos de ejecución (ms) cuando la tarea es muy breve.

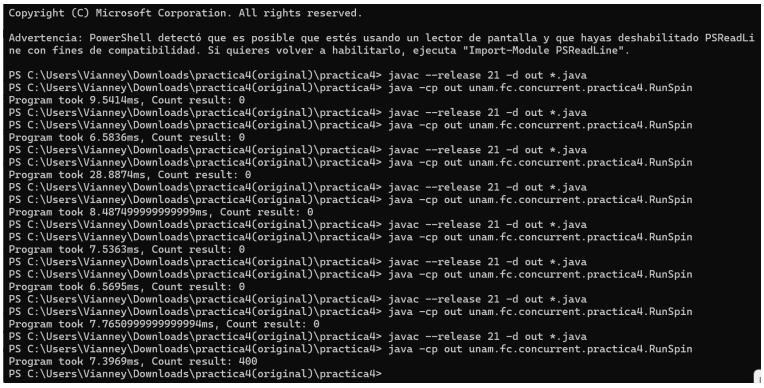


Figura 1: Ejecuciones de `RunSpin` con 400 tareas y 4 hilos.

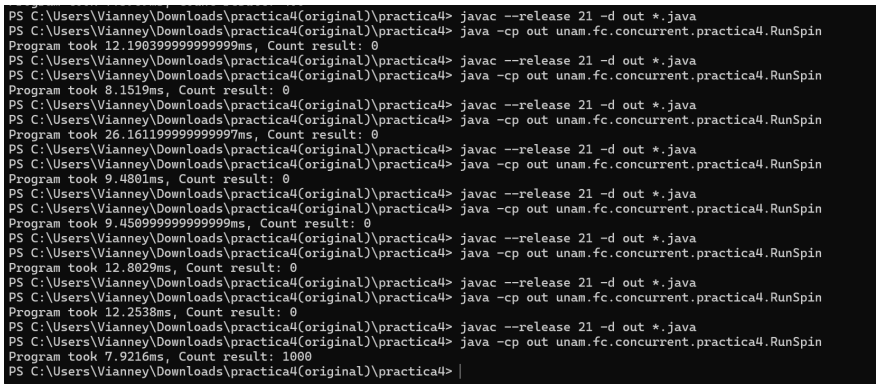


Figura 2: Ejecuciones de `RunSpin` con 1000 tareas y 4 hilos.

```
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> javac --release 21 -d out *.java
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> java -cp out unam.fc.concurrent.practica4.RunSpin
Program took 9.0215ms, Count result: 0
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> javac --release 21 -d out *.java
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> java -cp out unam.fc.concurrent.practica4.RunSpin
Program took 13.902099999999999ms, Count result: 0
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> javac --release 21 -d out *.java
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> java -cp out unam.fc.concurrent.practica4.RunSpin
Program took 34.2042ms, Count result: 0
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> javac --release 21 -d out *.java
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> java -cp out unam.fc.concurrent.practica4.RunSpin
Program took 10.545499999999999ms, Count result: 0
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> javac --release 21 -d out *.java
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> java -cp out unam.fc.concurrent.practica4.RunSpin
Program took 18.6391ms, Count result: 0
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> javac --release 21 -d out *.java
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> java -cp out unam.fc.concurrent.practica4.RunSpin
Program took 9.3971ms, Count result: 0
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> javac --release 21 -d out *.java
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> java -cp out unam.fc.concurrent.practica4.RunSpin
Program took 9.4831ms, Count result: 0
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> javac --release 21 -d out *.java
PS C:\Users\Vianney\Downloads\practica4(original)\practica4> java -cp out unam.fc.concurrent.practica4.RunSpin
Program took 9.2282ms, Count result: 1000
PS C:\Users\Vianney\Downloads\practica4(original)\practica4>
```

Figura 3: Ejecuciones de `RunSpin` con 1000 tareas y 11 hilos.

Ejercicio 2

Configuración	TAS	TTAS	Backoff	MCS	ALock	CLH	Reentrant
100 tareas, 4 hilos	120.24	107.32	154.78	123.91	121.94	115.98	106.25
100 tareas, 11 hilos	998.00	882.22	219.20	831.07	847.44	929.89	**110.85**

Cuadro 2: Tiempos de ejecución (ms) cuando la tarea es más pesada.

Como cada ejecución imprime 100 matrices acontinuación solo se muestra una captura de ejemplo con el final de la salida (`CLHLock` con 11 hilos). Se observa que cada matriz contiene únicamente valores de su propia tarea, que las tareas las ejecutan hilos distintos del *pool* y en la última línea el tiempo de ejecución.

```
  97020  97021  97022  97023  97024  97025  97026  97027  97028  97029
  97030  97031  97032  97033  97034  97035  97036  97037  97038  97039
  97040  97041  97042  97043  97044  97045  97046  97047  97048  97049
  97050  97051  97052  97053  97054  97055  97056  97057  97058  97059
  97060  97061  97062  97063  97064  97065  97066  97067  97068  97069
  97070  97071  97072  97073  97074  97075  97076  97077  97078  97079
  97080  97081  97082  97083  97084  97085  97086  97087  97088  97089
  97090  97091  97092  97093  97094  97095  97096  97097  97098  97099
Tarea 98 (pool-1-thread-11)
  98000  98001  98002  98003  98004  98005  98006  98007  98008  98009
  98010  98011  98012  98013  98014  98015  98016  98017  98018  98019
  98020  98021  98022  98023  98024  98025  98026  98027  98028  98029
  98030  98031  98032  98033  98034  98035  98036  98037  98038  98039
  98040  98041  98042  98043  98044  98045  98046  98047  98048  98049
  98050  98051  98052  98053  98054  98055  98056  98057  98058  98059
  98060  98061  98062  98063  98064  98065  98066  98067  98068  98069
  98070  98071  98072  98073  98074  98075  98076  98077  98078  98079
  98080  98081  98082  98083  98084  98085  98086  98087  98088  98089
  98090  98091  98092  98093  98094  98095  98096  98097  98098  98099
Tarea 99 (pool-1-thread-1)
  99000  99001  99002  99003  99004  99005  99006  99007  99008  99009
  99010  99011  99012  99013  99014  99015  99016  99017  99018  99019
  99020  99021  99022  99023  99024  99025  99026  99027  99028  99029
  99030  99031  99032  99033  99034  99035  99036  99037  99038  99039
  99040  99041  99042  99043  99044  99045  99046  99047  99048  99049
  99050  99051  99052  99053  99054  99055  99056  99057  99058  99059
  99060  99061  99062  99063  99064  99065  99066  99067  99068  99069
  99070  99071  99072  99073  99074  99075  99076  99077  99078  99079
  99080  99081  99082  99083  99084  99085  99086  99087  99088  99089
  99090  99091  99092  99093  99094  99095  99096  99097  99098  99099
Program took 929.8943999999999ms, 100 tareas con 11 hilos
PS C:\Users\Vianney\Downloads\practica4(original)\practica4>
```

Figura 4: Final de la salida de `RunSpinMatrix` con `CLHLock`, 100 tareas y 11 hilos.

3. Ejercicio 3

¿Cuáles son las 2 implementaciones más rápidas del inciso 1? ¿A qué crees que se debe?

Lass dos más rápidas fueron **CounterAtomic** (8.18 ms) y **MCSLock** (9.51 ms) (tomando el promedio de las tres configuraciones), pero el segundo lugar está prácticamente empatado con **TTASLock** (9.54 ms) y `ReentrantLock` (9.59 ms), pues la diferencia es de centésimas de milisegundo.

- **CounterAtomic** no usa candado: cada incremento es una sola instrucción atómica de hardware (`getAndIncrement`, que en x86 es un `lock xadd`). No hay fase de adquisición ni de liberación, ningún hilo espera a otro y la operación nunca se repite. Es la forma más barata de proteger una operación tan pequeña y por eso fue el más rápido o casi el más rápido en las tres configuraciones.
- **MCSLock** forma una cola en la que cada hilo gira sobre *su propio* nodo (`qnode.locked`) que vive en su caché local. Al salir el dueño le pasa el candado directamente a su sucesor escribiendo solo en el nodo de éste, y así no hay una variable compartida que todos estén leyendo y escribiendo al mismo tiempo y el tráfico de coherencia de caché es mínimo. Con 11 hilos en una máquina de 12 nunca hay más hilos que núcleos por lo que no aparece el problema típico de las colas (que el sistema operativo desaloje al siguiente de la fila).
- **TTASLock** (prácticamente empatado) espera *leyendo* `state` desde su caché y solo intenta el `getAndSet` cuando ve el candado libre, lo que genera muchas menos invalidaciones que `TASLock` que escribe en cada vuelta.

Como la tarea es tan breve (un `counter++`) casi todo el tiempo medido es el costo fijo de arrancar la JVM, crear el *pool* de hilos y enviar las tareas, que es igual para todos los candados, es por eso que los tiempos están tan juntos (entre 6 y 14 ms). La única que se separa claramente es **BackoffLock** (29.75 ms en promedio, la más lenta en las tres configuraciones): cuando falla, duerme con `Thread.sleep` entre 0 y 32 ms (y el límite se duplica hasta 1024 ms) mucho más de lo que dura la sección crítica.

4. Ejercicio 4

¿Cuáles son las 2 implementaciones más rápidas del inciso 2? ¿A qué crees que se debe?

Las dos más rápidas fueron **ReentrantLock** y **BackoffLock**. La diferencia se ve más con 11 hilos: ReentrantLock tardó 110.85 ms y BackoffLock 219.20 ms, mientras que todos los spinlocks puros tardaron entre 830 y 1000 ms, es decir, de 4 a 9 veces más.

Ahora la sección crítica es larga: llenar 100 casillas e imprimir la matriz en la consola (una operación de entrada/salida mucho más lenta que un incremento). Mientras un hilo está dentro los demás esperan mucho tiempo y lo que importa es *qué hacen mientras esperan*:

- **ReentrantLock** no gira: si no obtiene el candado el hilo se estaciona (`LockSupport.park`) y el sistema operativo le da el procesador a otro hilo en particular al que tiene el candado y al proceso de la consola que dibuja la salida. No se desperdicia CPU.
- **BackoffLock** se comporta parecido: al fallar duerme con `Thread.sleep` libera el procesador y deja de competir por la variable `state`. Con una sección crítica larga ese tiempo dormido ya no es un desperdicio porque de todos modos el candado iba a estar ocupado.

En cambio, en TAS, TTAS, MCS, ALock y CLH los 10 hilos que esperan siguen girando en un ciclo `while` y además el hilo `main` también gira en `while(!isDone())`. Eso son 11 o 12 hilos ocupando al 100 % los 12 hilos de hardware.El hilo que tiene el candado (y la consola que debe imprimir) tiene que competir por procesador con hilos que no hacen trabajo útil y todo se alarga. Por eso con 4 hilos (cuando todavía sobran núcleos) los siete candados tardan casi lo mismo (106 a 155 ms; ahí Backoff es incluso el más lento por sus esperas largas) pero con 11 hilos los spinlocks se disparan.

5. Ejercicio 5

¿Cuáles implementaciones satisfacen la propiedad de Justicia? (ignorando ReentrantLock)

Las que satisfacen justicia (*first-come-first-served*: si A termina su *doorway* antes de que B empiece la suya, A entra antes que B) son los candados de cola: **ALock**, **CLHLock** y **MCSLock**. En los tres, la *doorway* es una sola operación atómica sobre la cola (`tail`) (que termina en un número acotado de pasos) y esa operación fija el orden de entrada de los hilos:

- **ALock**: `tail.getAndIncrement()` le da a cada hilo un turno distinto en el mismo orden en el que llegaron (`slot` = `tail` mód `size`). Cada hilo gira sobre su propia casilla `flag[slot]` y quien sale pone `flag[slot]=false` y `flag[(slot+1) % size]=true`, es decir, le pasa el candado exactamente al siguiente que llegó. (Requiere que `capacity` sea al menos el número de hilos que compiten a la vez; por eso en `RunSpin` se crea con `numberThreads`.)
- **CLHLock**: `tail.getAndSet(qnode)` pone al hilo al final de una lista implícita y le devuelve a su predecesor. Cada hilo gira sobre `pred.locked` y solo entra cuando su predecesor libera (`locked=false`). Por lo tanto entran en el orden del `getAndSet`.
- **MCSLock**: `queue.getAndSet(qnode)` también forma una fila; el hilo se enlaza con `pred.next = qnode` y gira sobre su propio `qnode.locked`. Al salir, el dueño del candado pone `qnode.next.locked=false` y le pasa el candado directamente a su sucesor (si todavía no se ha enlazado, lo espera). Entran en orden de llegada.

En los tres, ningún hilo que llegó después puede adelantarse por lo que que también son libres de inanición: un hilo en la posición k de la fila entra después de a lo más k secciones críticas.

No son justos `TASLock`, `TTASLock` ni `BackoffLock`: todos los hilos compiten por el mismo `getAndSet(true)` sobre `state` y cuando el candado se libera lo gana el primero cuyo `getAndSet` llegue sin importar cuánto tiempo lleve esperando. Un hilo con mala suerte puede perder siempre (inanición). En `BackoffLock` es todavía más claro: un hilo que lleva mucho esperando suele estar dormido con un tiempo largo y un recién llegado le gana el candado. Esto se observó en el ejercicio 2: con Backoff, la última matriz impresa fue la de la tarea 26 (con 4 hilos) y la de la tarea 6 (con 11 hilos), mientras que con los demás candados las últimas fueron tareas del final (96–99).